PROJECT SENSEWALK

Final Project Report

What we built, what we measured, and what is still unproven

A low-cost smart assistive walker for visually impaired users: two ESP32-S3 boards (one that senses and brakes, one that sees and speaks), our own AI models, and the software, tests and tools around them. 27 October 2026.

Repository	github.com/sayansaunth-lang/sensewalk (169 files, 55 commits, GitHub checks green)
Real hardware used	2 x ESP32-S3 (16 MB flash, 8 MB PSRAM), GC2145 camera, 1 x VL53L0X distance sensor, laptop. Raspberry Pi 3B owned but never brought online
Software	Python (Pi/laptop side) 201 tests, firmware 22 unit tests + 2 host checks, all passing; 4 automatic GitHub jobs
Own AI models	YOLOv8n pothole detector (3,011,043 parameters, Pi/laptop) and a tiny pothole CNN (60,530 parameters) that runs on the camera board
Done on real hardware	Both boards flashed and running our firmware; camera + on-board AI running at about 2 frames per second; simulated-sensor safety test 9 of 9 checks
Not done	Physical walker, brake, most sensors, amplifier audio, board-to-board wiring, real-world accuracy, 50 ms brake timing

How to read this report. Every number was measured on a real run, and anything that was *not* tested is labelled as such. Section 9 lists exactly what has and has not been verified, so nobody on the team assumes more than is true.

1. Summary

SENSEWALK splits the work so that nothing safety-critical depends on a camera. A **safety ESP32-S3** reads distance, motion and grip sensors and is designed to brake by itself. A **camera ESP32-S3** runs two small AI models on its own, vibrates the walker through the safety board and speaks warnings through an amplifier. A Raspberry Pi or laptop is optional and adds the full-size AI, sign reading and the fall-alert SMS.

What is complete and tested

- **All the Pi/laptop software** (vision, sensor fusion, speech, emergency alert, UART protocol, training tools): 201 automated tests, re-run on a clean Linux machine by GitHub.
- **Both ESP32-S3 boards run our firmware.** The safety board boots, refuses to run without its sensors, and passes a simulated-sensor safety test on the real chip. The camera board runs the person and pothole AI at about 2 frames per second.
- **Our own pothole AI, twice:** a YOLOv8n detector for the Pi/laptop and a 60,530-parameter classifier for the camera board, whose C++ output matches PyTorch to 6.6e-07 on the real chip.
- **Voice alerts** from your five recorded clips, on the laptop/Pi and, for four of them, on the camera board.
- **Two real bugs in our own design were found only because we ran on hardware** (section 7): a pin map that crashed the ESP32-S3, and a message counter that reported false packet loss.

What is not yet proven

- **The walker itself:** no chassis, brake, grip or ultrasonic sensors, and only one (wrong-model) distance sensor. The brake logic has run only against simulated sensor values.
- **The 50 ms brake time** is a goal. The software decision step measures about 5 microseconds, but sensor-to-brake time has never been measured.
- **AI accuracy in the real world.** The pothole models were tested on held-out web photos only. A screen-based camera test was run but turned out invalid (the camera was not aimed at the screen), and it hinted that indoor scenes trigger false pothole alerts.
- **Board-to-board wiring and the amplifier.** The messages between the boards decode correctly in software; the physical 3-wire link and audio output have not been wired or heard.
- **The Raspberry Pi 3B** never came online, so the camera module and the Pi benchmark were never run.

> **The design rule that holds it together.** A camera cannot measure distance, so vision may only **warn** (speech and vibration). Only the safety board's time-of-flight and ultrasonic sensors can fire the brake. A slow, crashed or wrong camera therefore cannot cause a wrong brake decision.

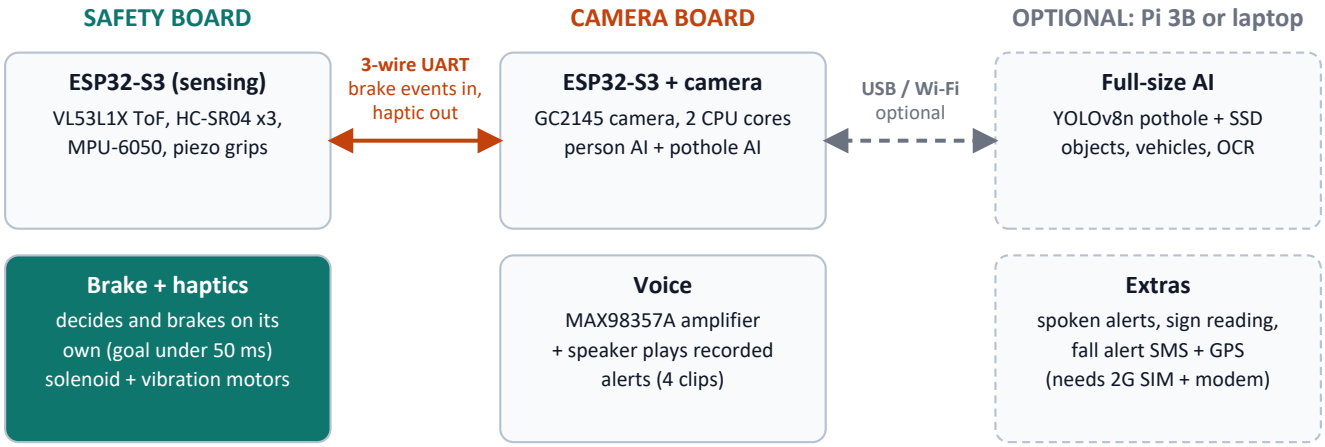
2. Status at a glance

Area	Status	Evidence / caveat
Pi/laptop software (vision, fusion, speech, emergency, comms)	**Done, tested**	201 tests pass; GitHub runs them on every push
Safety ESP32-S3 firmware	**Runs on the real chip**	Flashed; boots; halts with a clear message when sensors are missing; 22 unit tests; **never run with real sensors and a brake**
Simulated-sensor safety test	**Passed 9 of 9**	Real firmware decisions + UART against scripted sensor values; 3,311 frames, 0 checksum errors, 0 lost
Distance sensor	**1 sensor, wrong model**	A VL53L0X (about 1.2 m range) reads correctly; the design needs a VL53L1X pair. A one-sensor build exists (no drop-off detection)
Camera board (ESP32-S3 + GC2145)	**Runs on the real board**	Camera, person AI and pothole AI at about 2.1 fps; self-test passes; alert frames decode correctly
Person detector (camera board)	**Working, lightly tested**	Wall: 1 of 40 frames said person (0 confirmed). Partly visible head: 19 of 40. No proper hit-rate measured
Pothole detector (camera board)	**Working, accuracy unproven**	Held-out web photos: about 6% false alarms, about 71% caught. Real camera test not yet valid
Voice through the amplifier	**Code runs, audio unheard**	All 4 clips play with correct timing on the board; no amplifier attached yet
Fall alert: SMS with GPS	**Built, fake-modem tested**	Never run on a real SIM800L or GPS; needs a 2G network
Raspberry Pi 3B	**Not online**	Never found on any network; camera module never tested (section 8)
Thresholds (drop-off, obstacle, fall, grip, person margin)	**Placeholders**	Marked TODO(calibrate); must be replaced with bench measurements
Chassis, brake hardware, power, wiring	**Not started**	Mechanical/electrical track

Team roles (from the project plan)

Member	Role	Owns in the repository
M1	Chassis and ergonomics	mounts, camera angle
M2	Actuation and power	solenoid/MOSFET wiring, battery, fuse (`docs/WIRING.md`)
C1	Embedded firmware	`firmware/`
C2	Computer vision	`vision/pi/`, `training/`
C3	Integration and fusion	`comms/`, `fusion/`, `emergency/`
C4	Speech, test and docs	`speech/`, `test/`, docs, demo

3. How the system fits together



Why two boards? The safety loop must never wait for a camera or a neural network. The safety board runs a real-time operating system (FreeRTOS) with the brake loop at the highest priority and depends on nothing from the camera board. The camera board has the memory and two CPU cores to run the AI, and it can crash or restart without affecting braking.

The link between the boards

- **Three wires** (TX, RX, ground) at 115200 baud. Every line carries a sequence number and checksum. The camera board sends `haptic,150|150` style commands (person = 150, pothole = 255, clear = 0, repeated 5 times a second). The safety board sends `brake_state,1/0`, which the camera board turns into the obstacle and road-clear voice clips.
- **The safety board never depends on this link.** Unplugged, it keeps sensing and braking. It already understood these messages because they are the same protocol the Pi would use, so it needed no firmware change.

What each board decides

Priority	Input	Result	Can it brake?
1	Ground drop-off (downward distance sensor jumps)	Brake; voice "obstacle" clip	**Yes** (real distance)
2	Obstacle close (ultrasonic or forward ToF)	Brake; voice "obstacle" clip	**Yes** (real distance)
3	Pothole seen by the camera	Vibrate strongly + pothole voice clip	No, warning only
4	Vehicle seen by the camera (Pi/laptop only)	Vehicle / traffic voice clip	No, warning only
5	Person seen by the camera	Vibrate + person voice clip	No, warning only
Always first	Fall detected (IMU)	Fall alert countdown, then SMS (Pi/laptop side)	-

- **Brake release needs a hand on the grip.** The brake does not let go just because the hazard cleared; the grip sensors must confirm the user is holding on.
- **Noise filters at every layer:** a drop-off must appear in two consecutive readings, and each camera warning needs 2 of the last 3 frames to agree.
- **Known limit of the camera warnings:** they say something is in view, not where or how far.

4. What has been built

Part	What it does	Folder
Safety firmware	Reads ToF, ultrasonic, IMU and grip sensors; decides to brake on its own; drives the solenoid brake (5 s anti-overheat cutoff) and vibration motors; talks over UART. PlatformIO build (CI-tested) plus a flat Arduino IDE copy. Also: a simulated-sensor mode, a one-VL53L0X build, and bring-up sketches (blink, I2C scanner, sensor tests).	`firmware/esp32*`
Camera-board firmware	Camera capture, person AI (TensorFlow Lite Micro), pothole AI (plain C++), 2-of-3 confirmation, haptic commands, I2S voice playback, USB status and test commands.	`SENSEWALK_cam_main/`
UART protocol	The wire format between boards, implemented in Python and C++ and unit-tested on both sides.	`comms/`
Fusion state machine	Combines sensor and vision inputs into one state with a fixed priority order, incl. new vehicle/traffic warnings.	`fusion/`
Vision pipeline (Pi/laptop)	Webcam or Pi camera, people/object model, own YOLO pothole model, vehicle counting, OCR, frame skipping, simulation mode.	`vision/pi/`
Speech and voice clips	Fixed alert phrases, your 5 recorded clips (plus a stand-in person clip), non-blocking playback with priority, fallback to synthesised speech, buzzer watchdog.	`speech/`
Emergency alert	Fall starts a 10 s countdown that gripping cancels; otherwise an SMS with a map link, using GPS only if the fix is recent. Phone numbers come from an environment variable, never from code.	`emergency/`
Model training	Dataset cleaning, YOLOv8 training, compression, the tiny pothole CNN training/export, and evaluators that score the deployed code path.	`training/`
Test tools	Sensor logger, simulated-sensor checker, camera grabber/live viewer/screen test, field-test plan.	`test/`
Documentation	Architecture, wiring map (incl. camera board and amplifier), datasheet warnings, BOM, Pi setup, a guide to every file.	`docs/`

Quality measures built in

- **Automatic checks on every push** (GitHub, 4 jobs): the Python tests; a PlatformIO firmware build including the simulated and one-sensor variants; compilation of every Arduino sketch with the latest ESP32 software, publishing ready-to-flash files; and the firmware unit tests plus two host checks (the pothole network equals PyTorch; the voice decoder equals the reference).
- **Drift guards:** the generated voice header and the UART protocol copy must match their sources, or CI fails.
- **Safety by design:** unsafe or missing sensors stop the firmware with a clear message; phone numbers never live in code; datasets and personal text are never committed.

5. The AI models

Three neural networks are used. Two were trained by us; one is Google's. They run in different places because the camera board is a microcontroller and cannot run the big models.

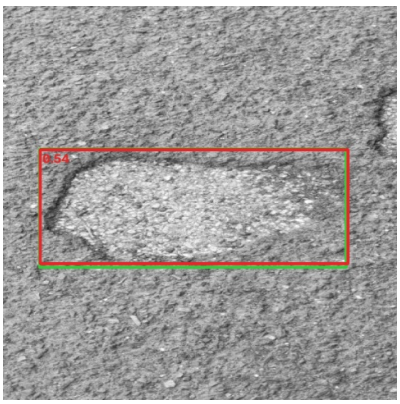
	Pothole detector (Pi/laptop)	Tiny pothole classifier (camera board)	Person model (camera board)
Who made it	**Us** (YOLOv8n fine-tune)	**Us** (small CNN from scratch)	Google (Visual Wake Words)
Parameters	3,011,043	60,530	about 0.3 MB model
Input	256 x 256, colour	64 x 64, grayscale	96 x 96, grayscale
Answers	*where* a pothole is (box)	is there a pothole in view (yes/no)	is there a person in view (yes/no)
Runs on	Pi / laptop (OpenCV)	ESP32-S3, plain C++	ESP32-S3, TensorFlow Lite Micro
Speed	about 12 ms laptop; Pi 3B unmeasured	412 ms alone	367 ms alone
Held-out accuracy	AP50 0.538, precision 0.68, recall 0.49	about 6% false alarms, about 71% caught (AUC 0.94)	not formally measured

Both vision models run in parallel on the camera board's two cores: about **2.1 frames per second** together (each is slower alone-in-parallel because they share memory). A separate people-and-objects model (Google SSD MobileNet, 4.2 MB) runs on the Pi/laptop and also finds vehicles.

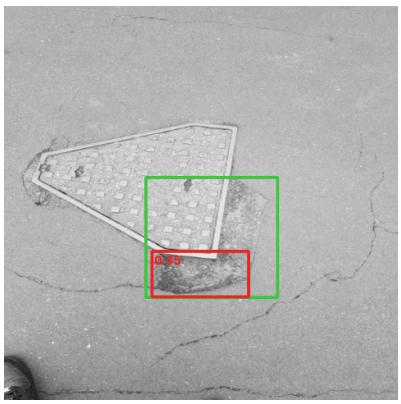
5.1 YOLOv8n pothole detector (Pi / laptop)

- **Data:** two public pothole datasets (CC BY 4.0 and MIT listings), cleaned and merged: 913 images kept after removing 101 exact duplicates and a few unusable ones. Split **730 train / 91 validation / 92 test** (1,959 labelled potholes). Near-duplicate images were clustered and kept in one split, so test scores are not inflated. A label file containing personal text was dropped and never copied to the repository.
- **Training:** fine-tuned on a laptop CPU at about 66 s per epoch; interrupted at epoch 24 of 60 when a session ended, best checkpoint (around epoch 17) used.
- **Compression:** FP16 weights halve the file (12.1 to 6.1 MB, no accuracy change, no RAM saving); a 256 px input does about 36% less computing with no loss detected. INT8 was not tried.

Version (92 held-out images)	Size	AP50	Precision	Recall	Hits / false alarms / misses
Original: 320 px, full precision	12.1 MB	0.511	0.64	0.49	91 / 51 / 96
256 px, FP16 (shipped)	**6.1 MB**	**0.538**	**0.68**	**0.49**	**92 / 44 / 95**



A hit. Green = labelled pothole, red = model (0.54).



A miss. The model's box covers only part of the labelled pothole, so it does not count.

These two were picked by rule (the first fully correct and the first fully missed test image), not chosen to flatter the model.

5.2 Tiny pothole classifier (camera board)

The YOLO model cannot run on an ESP32-S3, so we trained a much smaller network that answers one question: *does this view contain a pothole?* It is four stride-2 convolutions (16-32-64-64 channels), global average pooling and a 2-way output, written as 100 lines of plain C++ with no library.

- **Training views:** random square crops of the same leak-free photo split. A positive view mostly contains a labelled pothole; a negative view contains none (road, kerbs, sky, buildings from the same photos). Camera-like blur, noise and low contrast were added.
- **Held-out result** (257 views the model never saw: 180 pothole, 77 not): at the shipped threshold 0.73, about **6% of non-pothole views set it off and about 71% of pothole views are caught**; test AUC 0.94. The 77 negative views make each percentage uncertain by roughly 5 points. A smaller 15k-parameter version scored worse (AUC 0.89).
- **Proven on the chip:** 12 held-out views run through the C++ code on the real board and in CI match PyTorch to 6.6e-07, so the board computes what we trained.

> **What this does NOT show.** The test views are crops of web photos. The walker's camera is lower, in other light, and sees floors and pavements. The first camera-based test was invalid (the camera was not looking at the screen: it saw a curtain) but incidentally showed the detector scoring above its threshold in about **36% of frames on a patterned indoor scene**. Indoor surfaces were never taught as "not a pothole", so **expect false pothole alerts indoors** until indoor and real-camera examples are added to the training data.

5.3 Person model (camera board)

- Google's 96 x 96 person model runs through TensorFlow Lite Micro. It says "person in view" or not, never where or how far.
- **Measured so far (informal):** pointed at a plain wall it said person in 1 of 40 frames and confirmed 0; with part of a head at the frame edge it said person in 19 of 40. Two attempts at a whole-body test failed because the camera was aimed at the ceiling, so **a proper hit rate has not been measured**.

5.4 Weaknesses shared by the pothole models

- Small web-sourced data (about 900 photos, mostly daylight, mostly from standing or car height) and no clean-ground or indoor examples.
- One class only: stairs, curbs and open drains are not detected. 92 test images means true accuracy could differ by roughly 10 points.
- Licence: YOLOv8 is AGPL-3.0 (fine for a university project; decide before any distribution). The Google person model is Apache-2.0.

5.5 Voice alerts

Clip (your recording)	Says	Plays when
pothole_detected	"Alert. Pothole detected ahead. Reduce speed."	pothole confirmed (camera board, Pi/laptop) or distance-sensor drop-off
obstacle_warning	"Warning, obstacle detected ahead. Slow down."	the safety board reports a brake
road_clear	"Road ahead is clear."	the safety board reports the brake released
vehicle_ahead, traffic_ahead	"Caution. Vehicle detected ahead..." / "Traffic ahead..."	Pi/laptop only: a nearby vehicle / two or more vehicles
person_ahead (**stand-in**)	"Caution. Person ahead."	person confirmed; made with the Windows voice because no recording was supplied

Playback never blocks the vision loop, a more urgent clip interrupts a less urgent one, and a missing or failing clip falls back to synthesised speech. On the camera board the clips are stored as 16 kHz 8-bit mu-law (about 0.2 MB) and played through a MAX98357A amplifier. The brake clips say "obstacle" for any brake, because the safety board's message does not say why it braked.

6. Hardware bring-up: what actually happened

Step	Result
Safety board (ESP32-S3 N16R8)	Detected over USB (16 MB flash, 8 MB PSRAM). Toolchain download was too slow on this network, so GitHub now builds ready-to-flash files and we flash them with esptool
Blink test, then full firmware	Blink test runs. Full firmware boots, then stops with "VL53L1X ToF pair failed to initialize", as designed with no sensors attached
Wiring check (I2C scanner we wrote)	Found one device at 0x29. A chip-ID probe showed it is a **VL53L0X, not a VL53L1X**: same address, but the library rejects it
VL53L0X bench test	Reads a steady 105 to 125 mm and changes as the scene changes
Simulated-sensor mode	Scripted 40 s scenario against the real safety code and UART: brake engages for forward obstacle, drop-off and side obstacle; releases only with hands on grips; fall rule fires. **9 of 9 checks, 3,311 frames, 0 errors**
One-sensor build	A single VL53L0X as forward sensor (no drop-off detection). Boots and reads the real sensor
Camera board (ESP32-S3 N16R8)	Detected; camera found to be a **GC2145** (no JPEG engine) on the ESP32-S3-EYE pin layout; photos retrieved over USB
On-board AI	Person + pothole models run in parallel at about 2.1 fps; boot self-test matches PyTorch; alert frames decode with the Python codec
Speaker test	All four clips play with correct timing and no errors; **no amplifier attached, so audio was not heard**
Laptop webcam	Both Pi-side models run on the webcam at about 46 ms per frame and found a person

Bugs found and fixed (most only visible on real hardware)

Problem	How it was found	Fix
Pin map used original-ESP32 pins; GPIO26-35 are flash lines on the S3, GPIO19 is its USB pin: the board crashed in a reboot loop	Running simulated-sensor mode on the real board	Moved to safe pins; new automatic test blocks forbidden pins
UART message counter tracked per tag, but the ESP32 uses one counter: false packet loss on almost every message	Decoding real traffic from the board	Tracker now follows the single stream; new tests
Driving the unused sensor-enable pin low silenced the only sensor	One-sensor build on the real sensor	Both enable lines stay high
Sensor model mismatch (L0X vs L1X)	I2C chip-ID probe	Detected and documented; one-sensor build added
First TFLite library did not compile with current ESP32 software	GitHub build	Switched to a maintained port
Camera board ran out of fast RAM at boot	Boot message on the real board	Pothole workspace moved to PSRAM; smaller person arena
Running both models in turn gave only 1.2 fps	Timing on the board	Pothole model on the second core: 2.1 fps
Camera sensor has no hardware JPEG	Camera init error	Raw capture + on-chip JPEG compression
Screen test measured the wrong scene	Captured photo showed a curtain	Test now verifies the camera sees the screen or cancels
Earlier software: missing GPS/SMS code, OCR crash without Tesseract, SMS sent after modem ERROR, fall flag latched forever, firmware compile errors, UART length mismatch	Code audit, real runs, first GitHub builds	All fixed with regression tests

7. Testing and what it proves

What	How	Result	Proves / does not prove
Pi/laptop logic	201 automated tests on a clean Linux machine (GitHub)	All pass	Logic is correct; says nothing about the real walker
Firmware logic	22 unit tests + 2 host checks (pothole network vs PyTorch; voice decode)	All pass	Rules and codecs correct; not the hardware timing
Firmware compiles	Every sketch built with the latest ESP32 software in CI	Passes	It builds; run results are from the real boards
Safety logic on the chip	Simulated sensors, 40 s scenario, checker over USB	9 of 9	Real decision code + UART work; sensors and brake are simulated
Pothole models	92 held-out images / 257 held-out views	see section 5	Held-out web photos only; real-world accuracy unknown
Camera board	Boot self-test, live frames, alert frames decoded	Pass	It runs; accuracy not measured
Screen test of the pothole AI	Camera watching the laptop screen	**Invalid** (camera not aimed)	Nothing; rerun needed (script now checks the aim)

8. Raspberry Pi 3B: honest status

The plan was to run the full AI on a Pi 3B and test the Camera Module 3. **The Pi never came online where it could be reached**, so neither was done. We scanned the hotspot networks repeatedly: no Raspberry Pi hardware address ever appeared, the camera module was never tested, and the Pi benchmark was never run. The SD card could not be inspected because Windows gave its boot partition no drive letter and we had no administrator rights.

Most likely causes

- The SD card was written for a different Wi-Fi name or password than the network the laptop was on, or the Wi-Fi country was not set to IN, or the customisation settings were not applied.
- The Pi 3B supports 2.4 GHz only; a 5 GHz hotspot or a hostel network with a web login would never work. A weak power supply (needs 2.5 A) also causes flaky behaviour.
- **Fastest diagnosis:** plug in an HDMI monitor and keyboard, or use an Ethernet cable to a router.

> **Why this no longer blocks the project.** The camera board runs the person and pothole AI and the voice by itself, so a demonstration needs no laptop and no Pi. What the Pi would add: the more accurate YOLO pothole model, vehicle and traffic warnings, sign reading and the fall-alert SMS. None of that has been run on a Pi, and the Pi 3B's speed on these models is still unmeasured.

9. What was tested and what was not

Not verified (do not assume these work)

Item	Why it matters
Any real sensor except one VL53L0X	VL53L1X pair, ultrasonic, IMU, grips: wiring, addressing, readings and thresholds all untested
The brake and vibration motors	Solenoid driver, thermal cutoff and motors have never been connected
Brake within 50 ms	Only the software step (about 5 microseconds) is timed; needs a logic analyser or scope on the real brake
Camera board to safety board wiring	Messages decode in software; the three wires and the pins (a guess) are untested
Amplifier and speaker	Clips play with correct timing; nobody has heard them. Pins are a guess
AI accuracy on the walker's camera	No valid real-camera test yet; indoor false alarms likely

Item	Why it matters
Person detector hit rate	Only informal checks so far
Fall alert on real hardware	Real GPS fix, real SIM800L SMS and 2G service; only tested against a fake modem
Raspberry Pi 3B and Camera Module 3	Never online; speed and memory use unknown

10. Recommended next steps, in order

1. **Confirm the camera-board pins** (link TX/RX, amplifier BCLK/LRC/DIN) against its silkscreen, then wire the 3-wire link and the MAX98357A, and listen to the speaker test (send v over USB).
2. **Rerun the screen test** with the camera really aimed at the screen (the script now refuses to run otherwise). If indoor false alarms are confirmed, add indoor and real-camera frames as "not a pothole" and retrain with `training/tiny_pothole/`.
3. **Measure the person detector properly:** whole body at 1.5 m, empty room, walking across the frame; report hit and false-alarm rates.
4. **Buy the missing sensors** (list below) and wire them one at a time on the new pin map, using the bench tools. Replace the placeholder thresholds with measured values.
5. **Wire the brake driver** and prove the under-50 ms stop with a logic analyser, with the camera board unplugged.
6. **Film real ground footage** with the camera board at walker height (pavements, corridors, ramps, potholes, stairs) and label it; this is the slowest part of making the AI trustworthy.
7. **Bring the Pi up** with a monitor/keyboard or Ethernet if you want the full-size AI, vehicle warnings, OCR and the fall SMS; then run the on-device benchmark.
8. **Record your own "person ahead" clip** to replace the stand-in, and regenerate the board's voice header.

Parts still needed

Part	For	Note
VL53L1X distance sensors (2)	forward + ground drop-off sensing	the sensor in hand is a VL53L0X (short range)
HC-SR04 ultrasonic (3) + resistor dividers	side/forward obstacle sensing	echo pin is 5 V; divide to 3.3 V
MPU-6050, piezo grip discs	fall detection, brake release	
Solenoid, MOSFET module, flyback diode, vibration motors + transistors	brake and haptics	fuse the supply
MAX98357A amplifier + 4 to 8 ohm speaker (about 3 W)	voice alerts	powered from 5 V
Battery, buck converter, power bank for the camera board	power	camera board needs its own 5 V supply
GPS module, SIM800L + its own 2 A supply, 2 USB-serial adapters	fall alert SMS (Pi/laptop side)	SIM800L is 2G only

Appendix - Repository, commands, licences

Item	Detail
Repository	github.com/sayansaunth-lang/sensewalk; a guide to every file is `docs/FILE_GUIDE.md`; wiring is `docs/WIRING.md`
Flash a board	`python -m esptool --port COMx write-flash 0x0 .merged.bin` using the files GitHub publishes as the artifact `esp32s3-firmware`
Camera board settings	ESP32S3 Dev Module, PSRAM OPI, Flash 16 MB, Partition "16M Flash (3MB APP/9.9MB FATFS)" (the voice clips need it)
Safety simulator check	`python test/sim_check.py --port COM7 --seconds 45`
Camera tools	`test/cam_grab.py`, `cam_view.py`, `cam_person_check.py`, `cam_screen_test.py`
Regenerate voice clips	`python firmware/esp32_arduino_ide/tools/make_voice_header.py`
Person model	Google TensorFlow Lite Micro example (Visual Wake Words), Apache-2.0; people/object model: Google coco_ssd_mobilenet_v1 quant, Apache-2.0
Pothole base network	Ultralytics YOLOv8n, COCO-pretrained, AGPL-3.0
Pothole data	Hugging Face rupesh002/pothole-detection-dataset (MIT listing; a Roboflow export, original terms unverified) and Ryukijano/Pothole-detection-Yolov8 (Roboflow potholes-detection-d4rma v1, CC BY 4.0, attribution required)
Voice clips	Supplied by the team as MP3 (transcribed offline to match them to alerts); check you may publish them before making the repository public
Not in the repository	Raw datasets, training checkpoints, phone numbers (read from an environment variable)